LP-III · FINAL YEAR B.E. · PRACTICALS 15 – 19

Blockchain Technology

Every practical explained from the ground up, with worked examples and probable viva / oral questions with answers

+	**Blockchain and Ethereum basics** Before you start
15	**Installing MetaMask and studying the Ether spent per transaction** Practical 15
16	**Creating your own wallet with MetaMask** Practical 16
17	**Smart contract for a bank account** Practical 17
18	**Student data with structures, arrays and fallback** Practical 18
19	**Survey report on types of blockchain and their real-time use cases** Practical 19
+	**Quick revision and cross-practical questions** Revision

Based on the code in **github.com/qwerty7733/lp3**.

How to use: read the theory once, trace the worked example by hand, then cover the answers and test yourself on the questions. Answers are written the way you would say them in a viva — short first line, then the detail if the examiner digs deeper.

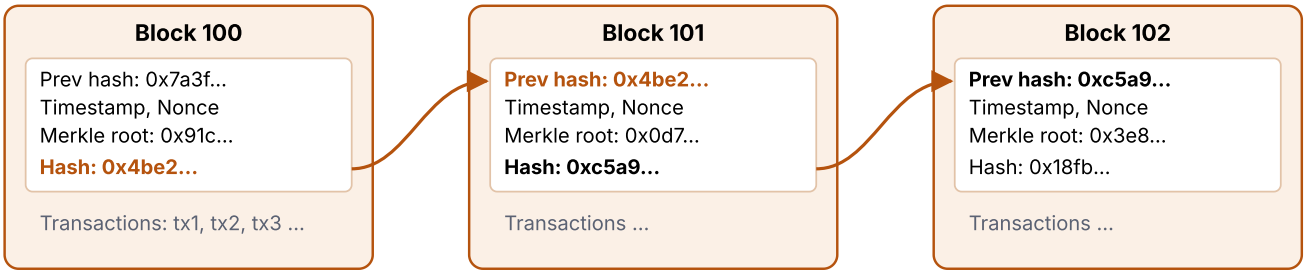
BEFORE YOU START

Blockchain and Ethereum basics

Practicals 15–19 build on each other: install a wallet (15, 16), write smart contracts and pay gas to run them (17, 18), then survey where blockchains are used (19). The ideas below come up in every BT viva.

What is a blockchain?

A **blockchain** is a **distributed, append-only ledger**: a list of records (transactions) grouped into **blocks**, where each block stores the **hash of the previous block**. Copies of the ledger are kept by many computers (**nodes**) that agree on the next block using a **consensus mechanism**, so no single party controls it.



Each block stores the previous block's hash. Changing any old transaction changes that block's hash, which breaks every link after it.

Why it is tamper-evident (immutable): a hash is a fingerprint of data. Change one character in block 100 and its hash changes completely, so block 101's "previous hash" no longer matches, and so on. An attacker would have to redo every later block faster than the rest of the network — practically impossible on a large chain.

Important terms

Term	Meaning
Hash function	turns any data into a fixed-size fingerprint; one-way, deterministic, tiny change → completely different output. Bitcoin uses SHA-256, Ethereum Keccak-256
Block header	previous hash, timestamp, nonce, Merkle root (and difficulty in PoW)
Merkle root	a single hash summarising all transactions in the block (from a binary tree of hashes); lets light clients verify one transaction without downloading the block
Nonce	in PoW: the number miners change to find a valid hash. In Ethereum accounts: a counter of transactions sent from an address
Node	a computer running the blockchain software and keeping a copy of the ledger
Consensus	the rule by which nodes agree on the next block — PoW, PoS, PBFT, PoA
Genesis block	the first block of a chain (block 0)
Smart contract	a program stored on the blockchain that runs automatically when called
DApp	decentralised application — a front end that talks to smart contracts
Gas	unit of computational work on Ethereum; fees are paid per unit of gas

Ethereum in one paragraph

Bitcoin is mainly a ledger of coin transfers. **Ethereum** is a programmable blockchain: besides transferring **Ether (ETH)**, anyone can deploy **smart contracts** that run on the **Ethereum Virtual Machine (EVM)** on every node. Contracts are usually written in **Solidity**. Ethereum switched from Proof of Work to **Proof of Stake** in **"The Merge" (September 2022)** and produces a block about every **12 seconds**.

Two kinds of accounts: an **Externally Owned Account (EOA)** is controlled by a private key (your MetaMask account); a **contract account** is controlled by its code and has no private key.

Ether units:

Unit	In wei	In ether	Used for
wei	1	10^{-18}	smallest unit; `msg.value` and balances in Solidity
gwei	10^{9}	10^{-9}	gas prices
ether	10^{18}	1	normal amounts

Warm-up questions

Q1. What is blockchain?

A decentralised, distributed digital ledger in which transactions are grouped into blocks that are linked by cryptographic hashes and replicated on many nodes, making the record transparent and very hard to alter.

Q2. Who invented blockchain / Bitcoin?

Bitcoin was proposed in 2008 in a white paper by the pseudonymous **Satoshi Nakamoto**, and the network launched in January 2009. Ethereum was proposed by **Vitalik Buterin** in 2013 and launched in 2015.

Q3. What are the main features of blockchain?

Decentralisation, immutability (tamper-evidence), transparency, security through cryptography, consensus without a central authority, and traceability of every transaction.

Q4. Why is blockchain called immutable?

Each block contains the hash of the previous block. Changing an old record changes its hash and breaks every later link, and the attacker would need to out-compute (PoW) or out-stake (PoS) the honest majority to rewrite history.

Q5. What is a hash function? Give its properties.

A function mapping any input to a fixed-size output. Properties: deterministic, fast to compute, pre-image resistant (cannot reverse it), collision resistant, and avalanche effect (small change → very different hash).

Q6. What is a consensus mechanism?

The method by which distributed nodes agree on the single valid version of the ledger and on the next block, even if some nodes are faulty or dishonest. Examples: PoW, PoS, PBFT, PoA.

Q7. Difference between Bitcoin and Ethereum?

Bitcoin is mainly digital money with a limited scripting language, uses PoW and SHA-256, and has ~10-minute blocks. Ethereum is a programmable platform with Turing-complete smart contracts on the EVM, uses PoS since 2022, Keccak-256, and ~12-second blocks.

Q8. What is the EVM?

The Ethereum Virtual Machine — the runtime on every Ethereum node that executes smart-contract bytecode deterministically, so all nodes get the same result.

Q9. What is a smart contract?

Code deployed on a blockchain that holds state and executes automatically when a transaction calls it. Its rules cannot be changed after deployment (unless designed to be upgradeable) and everyone can verify them.

Q10. What is the difference between an EOA and a contract account?

An EOA is controlled by a private key and can start transactions. A contract account is controlled by its code, has no private key and only acts when called.

PRACTICAL 15

Installing MetaMask and studying the Ether spent per transaction

Aim. Install MetaMask and study the spending of Ether per transaction.

Repo file: `BT_pract15_MetaMask_GasFee_STEPS.pdf` (no code — browser steps, screenshots and an observation table).

What is MetaMask?

MetaMask is a **non-custodial crypto wallet** available as a browser extension and mobile app. It stores your keys **locally, encrypted with your password**, shows your balances, signs transactions, and acts as the bridge between websites (**DApps**) and Ethereum or any EVM-compatible network (Polygon, BNB Chain, Sepolia, a local Hardhat/Ganache node...). It does not hold your coins — coins always live on the blockchain; the wallet holds the **keys** that control them.

Steps done in the practical: install from the official site → create wallet and password → back up the 12-word Secret Recovery Phrase → show test networks and select **Sepolia** → get free test ETH from a **faucet** → create Account 2 → send a small amount → inspect the fee in MetaMask and on **Etherscan** → repeat with Low and Aggressive fee settings.

Testnet vs mainnet: mainnet ETH has real value. A **testnet** such as Sepolia works the same way but its ETH is free (from faucets) and worthless, so you can practise safely. Older testnets like Ropsten, Rinkeby and Goerli have been shut down; Sepolia is the standard testnet for applications.

Gas — why every transaction costs Ether

Every operation in the EVM has a fixed **gas cost** (an addition 3 gas, storing a new value about 20,000 gas). Fees exist to:

- pay validators for processing and securing the network, and
- stop spam and infinite loops — every computation must be paid for, and a transaction runs out of gas instead of running forever.

Term	Meaning
Gas used	actual computation consumed; a plain ETH transfer always uses **21,000 gas**
Gas limit	maximum gas you allow the transaction to use. If it runs out, the transaction fails (reverts) but the gas used is still paid
Base fee	minimum price per gas set by the protocol for each block; rises when blocks are more than half full, falls when less. It is **burned** (destroyed)
Priority fee (tip)	extra per-gas amount paid to the validator to include your transaction sooner
Max fee	the most you are willing to pay per gas; anything above base + tip is refunded

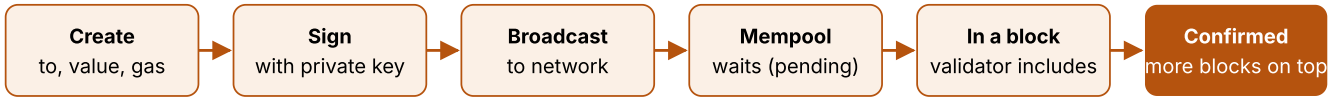
FEE FORMULA (EIP-1559, SINCE AUGUST 2021)

Transaction fee = Gas used × (Base fee + Priority fee)
Example: 21,000 gas × (12 gwei + 2 gwei) = 294,000 gwei = **0.000294 ETH**.
Total deducted from the sender = amount sent + fee.

Before EIP-1559 the fee was simply gas used × gas price, set by a blind auction. EIP-1559 made fees more predictable and burns the base fee, which reduces ETH supply.

Low / Market / Aggressive in MetaMask change the priority fee and max fee: a higher tip gets included faster; a low tip may wait longer when the network is busy. Gas used stays 21,000 for each setting — only the price per gas changes.

Life of a transaction



Fields to read on Etherscan: Transaction Hash, Status (Success / Fail), Block number, Timestamp, From, To, Value, Transaction Fee, Gas Price, Gas Limit & Usage, Base / Max / Priority fee, Burnt fees, and Nonce.

OBSERVATION TABLE FOR YOUR WRITE-UP

Fee setting	Gas used	Base fee (gwei)	Priority fee (gwei)	Fee paid (ETH)	Time to confirm
Low	21,000				
Market	21,000				
Aggressive	21,000				

Conclusion to write: gas used is fixed by the operation (21,000 for a transfer); the fee changes with the gas price, which depends on network demand and the tip chosen. A higher tip costs more but confirms faster.

Probable viva questions — Practical 15

Q1. What is MetaMask?

A non-custodial Ethereum wallet as a browser extension and mobile app. It manages accounts and keys, signs transactions, and connects websites (DApps) to Ethereum and other EVM networks.

Q2. What is a crypto wallet? Does it store coins?

Software or hardware that stores the private keys controlling blockchain accounts. It does not store coins — balances live on the blockchain; the wallet holds the keys that prove ownership and sign transactions.

Q3. What is gas?

The unit that measures how much computational work a transaction or contract call needs on Ethereum. Every EVM operation has a fixed gas cost.

Q4. Why do we need to pay gas fees?

To reward validators for processing transactions and to protect the network from spam and endless computations — every step of execution must be paid for.

Q5. How is the transaction fee calculated?

Fee = gas used × (base fee + priority fee). For example 21,000 × (12 + 2) gwei = 294,000 gwei = 0.000294 ETH.

Q6. How much gas does a simple ETH transfer use?

Exactly 21,000 gas.

Q7. What is the difference between gas limit and gas used?

Gas limit is the maximum the sender allows; gas used is what the transaction actually consumed. Unused gas below the limit is not charged.

Q8. What happens if the gas limit is too low?

The transaction runs "out of gas", all its state changes are reverted, but the gas consumed up to that point is still paid because validators did the work.

Q9. What is EIP-1559?

An Ethereum upgrade (London hard fork, August 2021) that introduced a protocol-set base fee per block, which is burned, plus an optional priority fee (tip) to validators. It made fees more predictable.

Q10. What is the base fee and why is it burned?

The minimum per-gas price for a block, adjusted automatically by demand (up when blocks are more than half full). Burning it removes ETH from circulation and stops validators from manipulating fees.

Q11. What is the priority fee (tip)?

An extra amount per gas paid directly to the validator to encourage faster inclusion of the transaction.

Q12. What is gwei? Convert 1 ETH into wei and gwei.

gwei = 10^9 wei, the unit used for gas prices. 1 ETH = 10^9 gwei = 10^{18} wei.

Q13. Why do gas prices change over time?

Block space is limited. When many users want to transact (popular NFT sale, market crash), the base fee rises; when the network is quiet it falls.

Q14. What is a testnet? Which testnet did you use?

A separate Ethereum network for testing, with free valueless ETH. We used Sepolia.

Q15. What is a faucet?

A website that gives free testnet ETH to an address so developers can pay gas on the testnet.

Q16. What is a mempool?

The pool of valid transactions waiting to be included in a block. A transaction shown as "pending" in MetaMask is in the mempool.

Q17. What is Etherscan?

A block explorer — a website to look up any Ethereum transaction, block, address or contract and see details such as status, fee, gas used and value.

Q18. What is a transaction hash?

A unique identifier of a transaction — the Keccak-256 hash of the signed transaction data. It is used to track the transaction on a block explorer.

Q19. What is the account nonce in a transaction?

A counter of how many transactions the sender has sent. It orders transactions from one account and prevents the same signed transaction from being replayed.

Q20. Can a confirmed transaction be cancelled?

No. A pending transaction can be "cancelled" or "sped up" by sending a new transaction with the same nonce and a higher fee, but once confirmed in a block it is final.

Q21. Why should you install MetaMask only from the official site?

Fake extensions and phishing sites imitate MetaMask to steal the recovery phrase. Anyone with the phrase can take all the funds.

PRACTICAL 16

Creating your own wallet with MetaMask

Aim. Create your own wallet using MetaMask for crypto transactions.

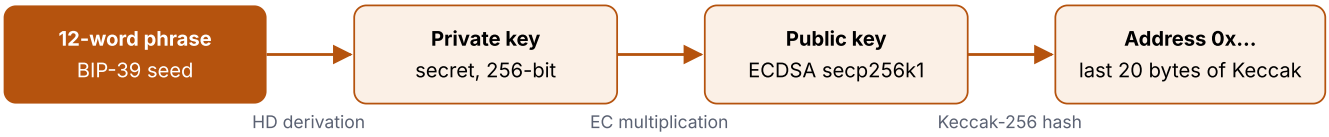
Repo file: `BT_pract16_CryptoWallet_STEPS.pdf` — create the wallet, explore the address and QR code, add a second account, get test ETH, send and receive, and read the transaction on Etherscan.

Public-key cryptography — the basis of every wallet

Each account has a **key pair**:

- **Private key** — a secret 256-bit number. Whoever knows it controls the account. It is used to **sign** transactions.
- **Public key** — calculated from the private key with elliptic-curve multiplication on the **secp256k1** curve. Easy one way, practically impossible to reverse.
- **Address** — the last **20 bytes** of the **Keccak-256 hash** of the public key, written in hex as **0x...** (42 characters). This is what you share to receive funds.

A **digital signature** (ECDSA) proves that the owner of the private key approved this exact transaction, without revealing the key. Every node can verify the signature using only public information.



Every arrow is one-way: you can go right easily but never back to the left.

Secret Recovery Phrase and HD wallets

The 12 words (**BIP-39 mnemonic**) are chosen from a fixed list of 2048 words and encode 128 bits of randomness plus a checksum. They generate a **master seed**, from which an **HD (hierarchical deterministic) wallet** derives any number of key pairs along a path such as **m/44'/60'/0'/0/0** (Account 1), .../0/1 (Account 2). That is why **one phrase restores every account** in MetaMask, and why "Add account" needs no new backup.

- **Password** — only unlocks MetaMask on *this* device; it cannot recover the wallet elsewhere.
- **Recovery phrase** — restores the wallet on any device. Never share it, never photograph it, never type it into a website.

Types of wallets

Classification	Types	Example
Who holds the keys	**Custodial** (exchange holds them) vs **non-custodial** (you hold them)	WazirX/Binance account vs MetaMask
Internet connection	**Hot** (online, convenient) vs **cold** (offline, safer)	MetaMask vs Ledger / Trezor hardware wallet, paper wallet
Form	software (desktop, mobile, browser), hardware, paper	—

"**Not your keys, not your coins**" — with a custodial wallet you trust the company; if it is hacked or fails, the funds may be lost.

What the transactions show

Sending 0.002 SepoliaETH from Account 1 to Account 2: Account 1's balance falls by **0.002 + fee**, Account 2 receives exactly **0.002**. The receiver pays nothing. The sending account's **nonce** goes up by one. On Etherscan the transaction shows its hash, block, from, to, value, fee and **Status: Success**.

Probable viva questions — Practical 16

Q1. What is a cryptocurrency wallet?

A tool that stores private keys and uses them to sign transactions, showing the balances of the associated addresses read from the blockchain.

Q2. What is the difference between a public key, a private key and an address?

The private key is a secret number used to sign. The public key is derived from it and used to verify signatures. The address is a short identifier derived by hashing the public key (last 20 bytes of Keccak-256), shared to receive funds.

Q3. Can someone find my private key from my address?

No. Hashing and elliptic-curve multiplication are one-way functions; reversing them would take longer than the age of the universe with current computers.

Q4. What is the Secret Recovery Phrase?

A 12-word (BIP-39) mnemonic that encodes the wallet's master seed. It can regenerate all private keys and accounts of the wallet on any device.

Q5. What is the difference between the MetaMask password and the recovery phrase?

The password only encrypts and unlocks the wallet on that device. The recovery phrase is the actual master secret that restores the wallet anywhere. Losing the password is fixable with the phrase; losing the phrase (and the device) means losing the funds.

Q6. What happens if someone gets your recovery phrase?

They can import the wallet on their device and transfer all funds out. Blockchain transactions are irreversible, so there is no way to get them back.

Q7. What is an HD wallet?

A hierarchical deterministic wallet: all key pairs are derived from one seed using a derivation path (BIP-32/BIP-44). One backup covers unlimited accounts.

Q8. What is a derivation path?

A standard route used to derive a key from the master seed, e.g. m/44'/60'/0'/0/0 — purpose 44 (BIP-44), coin type 60 (Ethereum), account 0, external chain 0, index 0.

Q9. Custodial vs non-custodial wallet?

Custodial: a third party (an exchange) keeps your keys — easy recovery, but you must trust them. Non-custodial: you keep the keys (MetaMask) — full control and full responsibility.

Q10. Hot wallet vs cold wallet?

A hot wallet is connected to the internet (MetaMask, mobile apps) — convenient but more exposed to hacks. A cold wallet keeps keys offline (hardware or paper wallet) — safer for long-term storage.

Q11. What is a digital signature and why is it needed?

A value created with the private key over the transaction data. Anyone can verify with the public key that the owner authorised exactly this transaction and that nothing was altered — without seeing the private key. Ethereum uses ECDSA on secp256k1.

Q12. Which cryptographic algorithms does Ethereum use?

ECDSA on the secp256k1 curve for keys and signatures, and Keccak-256 for hashing (addresses, transaction and block hashes).

Q13. Why does the receiver not pay any fee?

Only the sender initiates and signs the transaction, so the sender pays the gas.

Q14. Why did Account 1's balance fall by more than the amount sent?

Because the transaction fee (gas used × gas price) is deducted in addition to the value transferred.

Q15. What is a QR code in a wallet used for?

It encodes the public address so someone can scan it with a phone and send funds without typing the long address.

Q16. Why does the address have mixed upper and lower case letters?

EIP-55 checksum: the capitalisation pattern is derived from the address's hash, so a typo in a copied address can be detected by wallets.

Q17. What is token import in MetaMask?

Adding the contract address of an ERC-20 token so MetaMask reads and displays your balance of that token. The tokens already belong to the address on chain; import only makes them visible.

Q18. How does MetaMask connect to a DApp like Remix?

The DApp asks to connect (via the provider MetaMask injects into the web page, `window.ethereum`); after you approve, the site can read your address and request signatures, but every transaction still needs your confirmation in MetaMask.

PRACTICAL 17

Smart contract for a bank account

Aim. Write a smart contract for a customer's bank account with three operations: deposit money, withdraw money, and show balance.

Code: `BT_pract17_BankAccount.sol` (Solidity ^0.8.0), run in **Remix IDE** on Remix VM or on Sepolia through MetaMask.

Solidity building blocks used in this contract

Code	Meaning
`// SPDX-License-Identifier: MIT`	licence identifier; the compiler warns without it
`pragma solidity ^0.8.0;`	compile with version 0.8.x; 0.8 adds automatic overflow/underflow checks
`contract BankAccount { … }`	like a class; deployed to its own address on the chain
`mapping(address => uint256) private balances;`	a hash table: wallet address → balance in wei. Every key exists and defaults to 0
`address public owner;`	`public` state variables get an automatic getter function
`event Deposited(address indexed customer, uint256 amount);`	a log entry stored in the transaction receipt; `indexed` makes it searchable
`constructor()`	runs once at deployment; here sets `owner = msg.sender`
`msg.sender`	address that called the function
`msg.value`	amount of wei sent with the call
`payable`	function (or address) can receive Ether
`require(cond, "msg")`	if cond is false, revert the whole call with that error message
`emit`	writes an event
`view`	reads state but does not change it — free when called from outside
`address(this).balance`	Ether currently held by this contract

Visibility: `public` (anyone, inside and outside), `external` (only from outside), `internal` (this contract and child contracts), `private` (this contract only). Note: "private" data is still visible on the blockchain to anyone reading storage — it only stops other contracts from accessing it.

How each function works

deposit() — `payable`, so Ether arrives with the call. It checks `msg.value > 0`, adds it to `balances[msg.sender]`, and emits `Deposited`. The Ether itself is held in the contract's own balance.

withdraw(amount) — follows the **Checks-Effects-Interactions** pattern:

1. **Checks:** `require(amount > 0)` and `require(balances[msg.sender] >= amount, "Insufficient balance")`.
2. **Effects:** `balances[msg.sender] -= amount` — update state **first**.
3. **Interactions:** `payable(msg.sender).transfer(amount)` — only then send Ether out.

getBalance() — `view`, returns the caller's balance in wei. **getBankBalance()** returns the total Ether in the contract (sum of all customers' deposits).

The re-entrancy attack and why the order matters

If the contract sent Ether **before** reducing the balance, a malicious contract could receive the Ether in its fallback function and immediately call `withdraw` again — the balance is still unchanged, so it passes the check again and again, draining the contract. This is the **re-entrancy attack** that hit **The DAO in 2016** (about 3.6 million ETH), which led to the hard fork that split Ethereum and Ethereum Classic. Reducing the balance first makes the second call fail the `require`.

transfer vs send vs call:

Method	Gas forwarded	On failure
`transfer(x)`	2,300	reverts automatically
`send(x)`	2,300	returns false (must be checked)
`call{value: x}("")`	all remaining (adjustable)	returns false — current recommended way, combined with Checks-Effects-Interactions or a re-entrancy guard

Testing in Remix

Deposit: enter 1 in VALUE, unit **ether**, click `deposit` → `getBalance` shows **1000000000000000000** (1 ETH in wei). Withdraw 0.5 ETH = **500000000000000000** wei. Withdrawing more than the balance reverts with "Insufficient balance" — the state is unchanged, but on a real network the gas used is still paid. Deposits and withdrawals are **transactions** (they cost gas and appear in the console); `getBalance` is a **call** (free, no transaction).

Probable viva questions — Practical 17

Q1. What is Solidity?

A statically typed, contract-oriented, high-level language for writing smart contracts that compile to EVM bytecode. Its syntax is influenced by C++, JavaScript and Python.

Q2. What is Remix IDE?

A browser-based IDE for Solidity: you can write, compile, deploy and test contracts on a built-in simulated chain (Remix VM) or on a real network through MetaMask (Injected Provider).

Q3. What does `pragma solidity ^0.8.0;` mean?

The file must be compiled with a compiler version ≥ 0.8.0 and < 0.9.0. The caret allows patch and minor updates within 0.8.

Q4. What is a mapping?

A key–value store, like a hash table: `mapping(address => uint256)` maps each address to a number. All keys exist with a default value of 0, and mappings cannot be iterated or asked for their length.

Q5. What is `msg.sender` and `msg.value`?

`msg.sender` is the address that called the current function. `msg.value` is the amount of wei sent along with the call (only in payable functions).

Q6. What does `payable` mean?

A payable function can receive Ether with the call; a non-payable function rejects any Ether sent. `payable(addr)` converts an address so it can receive Ether through `transfer` / `send`.

Q7. What is the purpose of `require`?

To validate conditions. If the condition is false, execution stops, all changes in the call are reverted, and the error message is returned to the caller. Gas used up to that point is consumed; the unused gas is returned.

Q8. Difference between `require`, `revert` and `assert`?

`require` checks inputs and conditions (user errors) with a message. `revert` aborts unconditionally, used inside if-blocks or with custom errors. `assert` checks internal invariants that should never be false — a failure indicates a bug.

Q9. What is an event and why use `emit`?

Events write logs to the transaction receipt. They are cheap compared with storage and are used by front ends to listen for activity (e.g. show "Deposit successful"). `emit` triggers the event; `indexed` parameters can be filtered.

Q10. What is a `view` function? Does it cost gas?

A function that reads but does not modify state. Called from outside (as in Remix's blue buttons) it runs locally on a node without a transaction, so it is free. If called inside a state-changing transaction it does use gas.

Q11. Difference between `view` and `pure`?

`view` can read state variables but not change them. `pure` neither reads nor changes state — it uses only its inputs (e.g. a function that adds two numbers).

Q12. What is a constructor?

A special function that runs exactly once when the contract is deployed, used to initialise state — here it stores the deployer's address as `owner`.

Q13. Where is the deposited Ether actually stored?

In the contract's own account balance (`address(this).balance`). The mapping only records how much of that total belongs to each customer.

Q14. Why is the balance shown as 10000000000000000000?

Solidity works in wei, the smallest unit; 1 ETH = 10^{18} wei. There are no decimals in Solidity.

Q15. What is the re-entrancy attack? How does your code prevent it?

A malicious contract re-enters `withdraw` from its fallback function before the balance is updated, withdrawing repeatedly. The code prevents it by reducing the balance before sending Ether (Checks-Effects-Interactions); `transfer`'s 2,300-gas limit also blocks re-entry.

Q16. What is the Checks-Effects-Interactions pattern?

A safe ordering for functions: first validate inputs (checks), then update the contract's state (effects), and only at the end call external addresses or send Ether (interactions).

Q17. Why are integer overflows not a problem in this contract?

Since Solidity 0.8, arithmetic is checked by default: an overflow or underflow reverts the transaction. Earlier versions needed the SafeMath library.

Q18. Difference between a transaction and a call in Remix?

A transaction changes state, is signed, mined into a block, and costs gas (orange/red buttons). A call only reads data from a node, creates no block entry and is free (blue buttons).

Q19. The `owner` variable is set but never used. How could you use it?

Add an `onlyOwner` modifier to admin functions, for example to pause the bank, see all deposits, or close the contract. Practical 18 shows such a modifier.

Q20. Can a deployed smart contract be modified?

No, its code is immutable. To change logic you deploy a new contract, or use an upgradeable proxy pattern designed in advance.

Q21. Why are smart contracts useful for banking?

They enforce rules automatically and transparently without an intermediary, work 24×7, and every operation is permanently auditable on the blockchain.

PRACTICAL 18

Student data with structures, arrays and fallback

Aim. Write a Solidity program to store student data using structures, arrays and a fallback function. Deploy it on Ethereum and observe the transaction fee and gas values.

Code: `BT_pract18_StudentData.sol` — `struct Student`, dynamic array `Student[] public students`, `onlyOwner` modifier, `addStudent`, `getStudent`, `getStudentCount`, `fallback()` and `receive()`.

Structures

A **struct** groups related fields into one custom type:

```
struct Student { uint256 rollNo; string name; string branch; uint256 marks; }
```

Create one with `Student(1, "Yash", "IT", 85)` (positional) or with named fields `Student({rollNo: 1, name: "Yash", …})`.

Arrays

- **Dynamic array** `Student[] public students;` — size can grow with `push()`; `length` gives the count. Used here.
- **Fixed array** `uint[5] marks;` — size fixed at compile time.
- `students.push(Student(...))` appends; `students[i]` reads; out-of-range index reverts. The code also checks `index < students.length` with a clear message.
- Because `students` is `public`, Solidity creates a getter `students(uint index)` that returns the struct's fields.

Data locations

Location	Where it lives	Cost	Used for
storage	permanently on the blockchain	expensive (writing a new slot ≈ 20,000 gas)	state variables like `students`
memory	temporary, during the function call	cheap	function arguments and local copies, e.g. `Student memory s`
calldata	read-only input data of an external call	cheapest	parameters of external functions

`string memory _name` must state its location because strings and arrays are reference types.

Modifiers

```
modifier onlyOwner() { require(msg.sender == owner, "Only owner can add students"); _; }
```

A modifier is reusable code that runs before (or around) a function. The `_;` marks where the function body is inserted. `addStudent` uses it, so only the deployer can add records.

fallback() and receive()

Situation when someone sends a transaction to the contract	Function that runs
calldata empty, Ether sent	`receive()` (if it exists; otherwise `fallback()` if payable)
calldata matches no function	`fallback()`
calldata matches a function	that function

Both must be `external`; to accept Ether they must be `payable`. A contract with neither rejects plain Ether transfers. In the code they only emit events (`FallbackCalled`, `EtherReceived`), which you can see in the Remix logs. In Remix you test them under **Low level interactions**: CALLDATA `0x1234` → fallback; empty CALLDATA with 1 wei → receive.

Gas and transaction fee observations

In Remix each transaction shows **gas** (the limit), **transaction cost** (total gas used, including the 21,000 base cost and input data) and **execution cost** (gas used by the code itself). On Sepolia, Etherscan also shows the gas price and the fee in ETH.

Operation	Relative gas	Why
Deploy contract	highest	32,000 creation cost + storing the bytecode (~200 gas per byte) + constructor
`addStudent` (first)	high	writes several new storage slots (array length, struct fields, strings)
`addStudent` (later, longer name)	high, varies	longer strings need more storage slots and more input data
`fallback` / `receive`	low	only emits an event — logs are much cheaper than storage
`getStudent`, `getStudentCount`	**free** (call)	view functions read data without a transaction

Fee = gas used × gas price. In your table, write the real numbers from your run. Typical conclusion: *storage writes dominate gas cost, deployment is the most expensive operation, and reading data with view functions is free.*

Probable viva questions — Practical 18

Q1. What is a struct in Solidity?

A user-defined type that groups several variables (fields) of different types into one unit, like a record. Here Student has rollNo, name, branch and marks.

Q2. What is the difference between a dynamic and a fixed-size array?

A dynamic array (`Student[]`) can grow with `push` and its length changes at runtime. A fixed-size array (`uint[5]`) has a length set at compile time.

Q3. How do you add an element to a dynamic storage array?

With `push`, e.g. `students.push(Student(1, "Yash", "IT", 85));`. The new element's index is `students.length - 1`.

Q4. What are storage, memory and calldata?

Storage: permanent contract state on the blockchain, expensive to write. Memory: temporary data for the duration of a call. Calldata: read-only, non-modifiable input data of external calls, the cheapest.

Q5. Why does `getStudent` copy the struct into memory?

`Student memory s = students[index];` makes a temporary copy so its fields can be returned; reading into memory is cheaper than reading each storage field repeatedly and makes clear that it is not modified.

Q6. What is a modifier? Explain `_;`.

A reusable piece of code attached to functions, usually for access control or validation. `_;` is the placeholder where the body of the modified function is executed.

Q7. What is a fallback function? When is it called?

A special unnamed function `fallback() external [payable]` that runs when a call's data does not match any function in the contract (or when Ether is sent with data and no other function applies).

Q8. What is the receive function? How is it different from fallback?

`receive() external payable` runs when Ether is sent with empty calldata (a plain transfer). `fallback` handles calls whose data matches no function. If `receive` is missing, a plain Ether transfer goes to a payable fallback.

Q9. Can a contract have more than one fallback function?

No — at most one `fallback` and one `receive` per contract, and neither can have arguments or return values (fallback may take and return bytes in its advanced form).

Q10. What happens if a contract has no receive or payable fallback and someone sends Ether?

The transfer reverts — the contract cannot accept plain Ether.

Q11. What is gas used vs transaction cost vs execution cost in Remix?

Transaction cost is the total gas the transaction used (the 21,000 base, data cost, plus execution). Execution cost is the part consumed by running the contract code. "Gas" in the console is the gas limit that was set.

Q12. Why did deployment cost the most gas?

Creating a contract has a fixed 32,000-gas cost plus a charge for every byte of bytecode stored on chain, plus the constructor's storage writes.

Q13. Why does adding a student cost a lot of gas?

It writes new values to storage — the array length, two numbers and two strings — and each new storage slot costs about 20,000 gas.

Q14. Why are view functions like getStudentCount free?

When called externally they are executed locally by one node and do not create a transaction or change state, so no gas is charged.

Q15. Why are strings expensive in Solidity?

They are dynamically sized; long strings take several 32-byte storage slots. To save gas, store short strings, use `bytes32`, or store data off-chain (IPFS) and keep only its hash on chain.

Q16. Is it a good idea to store student data on a public blockchain?

Not personal data in plain form — everything on a public chain is visible to everyone forever. Better: store hashes or encrypted data, or use a private/permissioned blockchain. It is good for tamper-proof certificates and marks verification.

Q17. What does the `onlyOwner` modifier protect against?

It stops any address other than the deployer from adding fake student records.

Q18. What is the difference between events and storage?

Storage is contract state that other contracts can read, and it is expensive. Events are logs in the transaction receipt, much cheaper, readable by off-chain applications but not by contracts.

PRACTICAL 19

Survey report on types of blockchain and their real-time use cases

Aim. Write a survey report on the types of blockchain and their real-time use cases.

Repo file: `BT_pract19_BlockchainSurvey_STEPS.pdf` gives the report structure (introduction, structure of a block, types, consensus, comparison table, use cases, advantages and limitations, conclusion, references). This chapter gives you the content to write it in your own words and answer viva questions on it.

Types of blockchain

Public (permissionless). Anyone can join, read, send transactions and take part in consensus. Fully decentralised and transparent, but slower and more expensive per transaction. *Examples:* Bitcoin, Ethereum, Solana.

Private (permissioned). One organisation controls who can join and validate. Fast, private and cheap, but centralised — users must trust the owner. *Examples:* Hyperledger Fabric deployments inside one company, internal audit or supply-chain ledgers.

Consortium (federated). A group of organisations shares control; only member nodes validate. Combines trust between members with efficiency. *Examples:* R3 Corda networks used by banks, Energy Web Chain for the energy sector, inter-bank trade-finance networks.

Hybrid. Mixes a private network (for confidential data) with a public chain (for verification — e.g. anchoring hashes on a public chain). Flexible, but more complex to design. *Examples:* XinFin (XDC), Dragonchain.

Feature	Public	Private	Consortium	Hybrid
Who can join	anyone	invited only	member organisations	mixed
Controlled by	nobody (community)	one organisation	group of organisations	organisation + public layer
Decentralisation	high	low	medium	medium
Speed / throughput	low	high	high	medium–high
Data visibility	fully transparent	private	shared among members	selective
Typical consensus	PoW, PoS	PoA, Raft, PBFT	PBFT, voting	combination
Examples	Bitcoin, Ethereum	Hyperledger Fabric	R3 Corda, Energy Web	XinFin, Dragonchain

Consensus mechanisms

Mechanism	How the next block is chosen	Used by	Pros / cons
Proof of Work (PoW)	miners race to find a nonce giving a hash below a target	Bitcoin	very secure; huge energy use, slow
Proof of Stake (PoS)	validators lock ("stake") coins and are chosen to propose blocks; misbehaviour is punished by **slashing**	Ethereum (32 ETH per validator), Cardano	~99.9% less energy than PoW; "rich get richer" concern
Delegated PoS	token holders vote for a small set of delegates	EOS, TRON	fast; more centralised
PBFT	known validators vote in rounds; tolerates f faulty nodes out of 3f + 1	Hyperledger Sawtooth, permissioned networks	fast finality; works only with a small known set of nodes
Proof of Authority (PoA)	pre-approved identified validators sign blocks	private/test networks	very fast; relies on trusting validators
Raft	a leader orders transactions (crash fault tolerant, not Byzantine)	Hyperledger Fabric ordering service	simple and fast; assumes nodes are honest

Real-time use cases

Sector	Use case	Example and type
Finance / payments	cross-border payments, remittances	RippleNet; stablecoins such as USDC on public chains
Decentralised finance	lending, exchanges without a bank	Uniswap, Aave (public – Ethereum)
Supply chain	track food and goods from farm to shop	IBM Food Trust on Hyperledger Fabric (Walmart traced mangoes in seconds instead of days) — private/consortium
Trade logistics	shipping documents	TradeLens by Maersk and IBM — **shut down in 2023** for lack of industry adoption, a useful lesson that technology alone is not enough
Healthcare	tamper-proof medical records, drug traceability	Estonia's e-health records secured with a blockchain-based integrity system; MediLedger for pharma
Digital identity & certificates	verifiable degrees and IDs	blockchain-anchored certificates that employers can verify instantly
Government	land records, voting, subsidies	land-registry pilots in several countries and Indian states
Energy	peer-to-peer energy trading, renewable certificates	Energy Web Chain (consortium)
NFTs & digital ownership	art, game items, tickets	ERC-721 tokens on Ethereum (public)
Insurance	automatic claim payout from data feeds	parametric insurance using oracles

Advantages and limitations

Advantages: decentralisation (no single point of failure), immutability and audit trail, transparency, security through cryptography, faster settlement without middlemen, programmable automation with smart contracts.

Limitations: limited scalability (Bitcoin ≈ 7 tx/s, Ethereum L1 ≈ 15–30 tx/s versus thousands for card networks), energy use of PoW, fees that rise with congestion, irreversibility of mistakes, privacy problems on public chains, smart-contract bugs, unclear regulation, and the **oracle problem** (a blockchain cannot itself verify real-world data).

Blockchain trilemma: it is hard to get **decentralisation, security and scalability** all at once. **Layer-2** solutions (rollups such as Arbitrum, Optimism, zkSync; Bitcoin's Lightning Network) and sharding try to improve scalability without giving up the other two.

Probable viva questions — Practical 19

Q1. What are the types of blockchain?

Public (permissionless), private (permissioned), consortium (federated) and hybrid.

Q2. Difference between a public and a private blockchain?

Public: open to anyone, fully decentralised and transparent, slower (Bitcoin, Ethereum). Private: one organisation controls access and validation, fast and confidential but centralised (Hyperledger Fabric).

Q3. What is a consortium blockchain? Give an example.

A permissioned blockchain governed by a group of organisations rather than one, such as banks sharing a trade-finance ledger. Examples: R3 Corda networks, Energy Web Chain.

Q4. What is a hybrid blockchain?

A combination where sensitive data stays on a private network while proofs or selected data are recorded on a public chain for verification. Examples: XinFin (XDC), Dragonchain.

Q5. What is a permissioned vs permissionless blockchain?

Permissionless: anyone can join and validate without approval. Permissioned: participants and validators need approval from an authority or consortium.

Q6. Explain Proof of Work.

Miners repeatedly change the nonce and hash the block header until the hash is below a target (difficulty). The first to find it broadcasts the block and earns the reward. Changing history would require redoing that work, which secures the chain.

Q7. Explain Proof of Stake.

Validators lock up (stake) coins as collateral. The protocol randomly selects validators to propose and attest to blocks, roughly in proportion to stake; dishonest behaviour causes part of the stake to be destroyed (slashing). It uses far less energy than PoW.

Q8. Why did Ethereum move from PoW to PoS?

To cut energy consumption by about 99.9%, lower the barrier to securing the network, and prepare for scalability upgrades. The switch, "The Merge", happened in September 2022.

Q9. What is a 51% attack?

If one party controls more than half of the mining power (PoW) or stake (PoS), it can reorder or reverse its own recent transactions (double spending) and censor others. It cannot steal other people's coins or forge their signatures.

Q10. What is Byzantine fault tolerance?

The ability of a distributed system to reach agreement even when some nodes are faulty or malicious (the Byzantine Generals problem). PBFT tolerates up to f faulty nodes out of 3f + 1.

Q11. What is Hyperledger Fabric?

An open-source permissioned blockchain framework hosted by the Linux Foundation, designed for enterprises. It has pluggable consensus (Raft ordering), private channels and smart contracts called chaincode (Go, Java, JavaScript).

Q12. Give real-life use cases of blockchain.

Cross-border payments, DeFi, supply-chain traceability (IBM Food Trust), medical records and drug tracing, digital certificates and identity, land records, voting, energy trading, NFTs and parametric insurance.

Q13. Is blockchain suitable for every application?

No. It suits cases where several parties who do not fully trust each other need a shared, tamper-proof record. If one trusted party controls the data or high speed and privacy are critical, a normal database is simpler and better.

Q14. What is the blockchain trilemma?

The difficulty of achieving decentralisation, security and scalability at the same time; improving one usually weakens another.

Q15. What are Layer-2 solutions?

Systems built on top of a blockchain (Layer 1) that process transactions off-chain or in batches and settle the result on the main chain — rollups (Optimistic, ZK) for Ethereum and the Lightning Network for Bitcoin. They increase speed and reduce fees.

Q16. What is the oracle problem?

Smart contracts can only see data on the blockchain. Real-world data (prices, weather) must be supplied by oracles such as Chainlink, and the contract is only as trustworthy as that data source.

Q17. What are the limitations of blockchain?

Low throughput, high or variable fees, energy use (PoW), irreversible errors, public-data privacy issues, smart-contract vulnerabilities, regulatory uncertainty and the need for oracles for real-world data.

Q18. What is a DApp?

A decentralised application whose back-end logic runs in smart contracts on a blockchain and whose front end connects through a wallet like MetaMask — for example Uniswap.

REVISION

Quick revision and cross-practical questions

One-page summary

Practical	What you did	Key concepts
15 MetaMask & gas	installed wallet, used Sepolia, sent ETH with different fee settings	gas, gas limit, base fee, tip, EIP-1559, gwei, faucet, Etherscan
16 Own wallet	created wallet, multiple accounts, sent and received	private/public key, address, seed phrase, HD wallet, signatures, custodial vs non-custodial
17 Bank account contract	deposit, withdraw, balance in Solidity	mapping, payable, msg.sender/value, require, events, view, Checks-Effects-Interactions, re-entrancy
18 Student data contract	struct + array storage, fallback, gas observation	struct, dynamic array, storage/memory/calldata, modifier, fallback vs receive, transaction vs execution cost
19 Survey	types of blockchain and use cases	public/private/consortium/hybrid, PoW/PoS/PBFT/PoA, trilemma, use cases

Mixed questions

Q1. Which of your practicals cost gas and which were free?

Deploying contracts, deposit, withdraw, addStudent, fallback/receive calls and all Ether transfers cost gas. View functions like getBalance, getStudent and getStudentCount were free calls.

Q2. Where did you use the concept of msg.sender in both contracts?

In BankAccount to credit and debit the caller's own balance; in StudentData to record the owner in the constructor and to restrict addStudent through the onlyOwner modifier.

Q3. Remix VM vs Injected Provider – MetaMask?

Remix VM is a simulated blockchain inside the browser with free test accounts — instant and no real network. Injected Provider uses MetaMask to deploy on a real network such as Sepolia, with real (test) gas fees, block confirmations and Etherscan records.

Q4. Why do failed transactions still cost gas?

Validators executed the code up to the point of failure; that computation must be paid for, even though all state changes are reverted.

Q5. What is the role of MetaMask when you deploy from Remix?

Remix creates the deployment transaction; MetaMask shows the fee, signs it with your private key and broadcasts it to Sepolia.

Q6. Which type of blockchain would you choose for a college marks/certificate system and why?

A consortium or hybrid chain — universities and employers as members, student data kept private, and certificate hashes anchored publicly so anyone can verify a certificate is genuine.